## 1. Proposed Architecture for SriToysRus Ltd (STRU) Online Platform

### a. Layers/Tiers of the Architecture

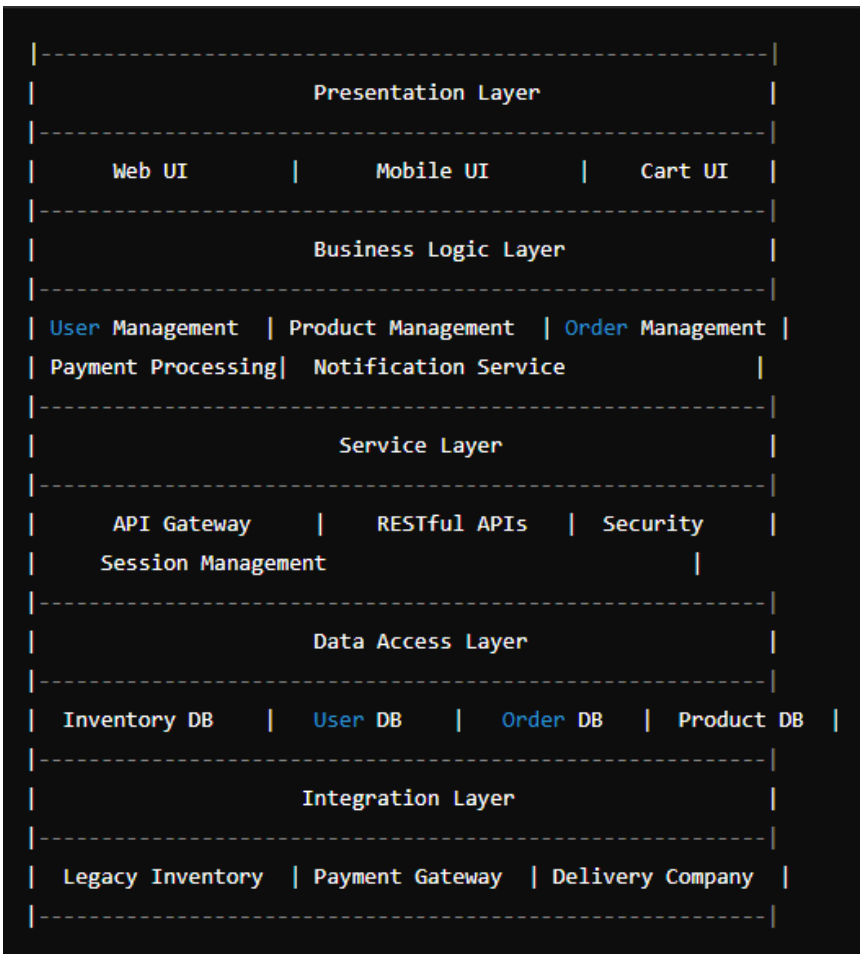
1. **Presentation Layer**
2. **Business Logic Layer**
3. **Service Layer**
4. **Data Access Layer**
5. **Integration Layer**

### b. Components of Each Layer/Tier

6. **Presentation Layer**
    - Web UI
    - Mobile UI
    - User Account Management
    - Shopping Cart Interface
7. **Business Logic Layer**
    - User Management Service
    - Product Management Service
    - Order Management Service
    - Payment Processing Service
    - Notification Service
8. **Service Layer**
    - API Gateway
    - RESTful APIs
    - Security (Authentication & Authorization)
    - Session Management
9. **Data Access Layer**
    - Inventory Database
    - User Database
    - Order Database
    - Product Database
10. **Integration Layer**
    - Legacy Inventory System
    - Payment Gateway
    - Delivery Company API

- External Logging/Monitoring Service

### c. Overall Architecture Sketch




### 2. Architectural Patterns Used in the Proposed Architecture

- **Layered Architecture**: Separates the application into layers, each responsible for specific functionality.
- **Microservices Architecture**: Each service (user, product, order, payment) is a separate, independently deployable component.
- **API Gateway Pattern**: Acts as a single entry point for all clients and routes requests to the appropriate microservices.
- **Repository Pattern**: Manages data access logic, ensuring a clean separation between business logic and data access.
- **Event-Driven Architecture**: For asynchronous communication between services.

**3. Legacy System Integration**

The legacy system in this scenario is the existing computerised inventory management system. This system needs to be integrated to ensure real-time updates of inventory across online and offline channels.

**Method of Integration**:

- **RESTful API Wrappers**: Create RESTful APIs around the legacy system to expose its functionalities to the new platform.
- **Message Broker (e.g., RabbitMQ, Kafka)**: Use a message broker to handle asynchronous communication between the legacy system and the new platform for real-time inventory updates.

**4. High-Level Sub-Systems of the Proposed Architecture**

11. User Management Sub-System
12. Product Management Sub-System
13. Order Management Sub-System
14. Payment Processing Sub-System
15. Notification Sub-System
16. Integration Sub-System (Legacy System, Payment Gateway, Delivery API)
17. API Gateway Sub-System
18. Session Management Sub-System
19. Logging/Monitoring Sub-System

**5. Asynchronous Communication Scenario**

**Interaction/Integration Suitable for Asynchronous Communication**:

- **Order Processing and Inventory Update**: When a user places an order, the system needs to update the inventory. This interaction can be done asynchronously to ensure the order process isn't delayed while waiting for the inventory update to complete.

**Justification**:

- **Resilience**: Asynchronous communication allows the system to handle failures gracefully. If the inventory update fails, the system can retry without affecting the user experience.

- **Scalability**: Asynchronous communication helps in handling high loads efficiently, as it decouples the order processing from inventory updating.
- **Responsiveness**: Users experience faster response times since the order process isn't waiting for the inventory update to complete.

Using a message broker like RabbitMQ or Kafka for this purpose can ensure reliable message delivery and retry mechanisms.